

The Complexity of Finding Coset-Generating Polymorphisms and the Promise Metaproblem

Manuel Bodirsky 

Institut für Algebra, TU Dresden, Germany

Armin Weiß 

FMI, University of Stuttgart, Germany

Abstract

We show that the metaproblem for coset-generating polymorphisms is NP-complete, answering a question of Chen and Larose: given a finite structure, the computational question is whether this structure has a polymorphism of the form $(x, y, z) \mapsto xy^{-1}z$ with respect to some group; such operations are also called *coset-generating*, or *heaps*.

Furthermore, we introduce a promise version of the metaproblem, parametrised by two polymorphism conditions Σ_1 and Σ_2 and defined analogously to the promise constraint satisfaction problem. We give sufficient conditions under which the promise metaproblem for (Σ_1, Σ_2) is in P and under which it is NP-hard. In particular, the promise metaproblem is in P if Σ_1 states the existence of a Maltsev polymorphism and Σ_2 states the existence of an abelian heap polymorphism – despite the fact that neither the metaproblem for Σ_1 nor the metaproblem for Σ_2 is known to be in P. We also show that the creation-metaproblem for Maltsev polymorphisms, under the promise that a heap polymorphism exists, is in P if and only if there is a uniform polynomial-time algorithm for CSPs with a heap polymorphism.

2012 ACM Subject Classification Theory of computation

Keywords and phrases constraint satisfaction problem, coset-generating polymorphisms, metaproblem, heap, abelian heap, uniform polynomial-time algorithm, NP-hardness

Digital Object Identifier 10.4230/LIPIcs.ICALP.2026.169

Category Track B: Automata, Logic, Semantics, and Theory of Programming

Related Version *Previous Version:* <https://arxiv.org/abs/2602.00778>

Funding *Manuel Bodirsky:* Received funding from the ERC (Grant Agreement no. 101071674, POCOCOP). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. *Armin Weiß:* Supported by the German Research Foundation (DFG) grant WE 6835/1-2.

1 Introduction

After the resolution of the finite-domain constraint satisfaction problem (CSP) dichotomy conjecture [32] (announced independently by Bulatov [12] and by Zhuk [34]), one of the central open problems in the area is whether there exists a *uniform* polynomial-time algorithm for tractable CSPs, i.e., an algorithm that decides in polynomial time whether there exists a homomorphism from a given relational structure $\mathfrak{A}$ to a given relational structure $\mathfrak{B}$, under the assumption that $\text{CSP}(\mathfrak{B})$ is in P. Here the structures are given by lists of tuples, that is, one list of tuples for each of the relations. Neither Bulatov’s nor Zhuk’s algorithm runs uniformly in polynomial time, because the running time of these algorithms depends superpolynomially on the size of the input structure $\mathfrak{B}$.

The existence of a uniform polynomial-time algorithm for tractable CSPs is already open under the much stronger assumption that the given structure $\mathfrak{B}$ has a *Maltsev polymorphism*, i.e., a polymorphism $m: B^3 \rightarrow B$ satisfying $m(x, x, y) = m(y, x, x) = y$ for all $x, y \in B$.



© Manuel Bodirsky and Armin Weiß;
licensed under Creative Commons License CC-BY 4.0

53rd International Colloquium on Automata, Languages, and Programming (ICALP 2026).
Editors: Sayan Bhattacharya, Danupon Nanongkai, Michael Benedikt, and Gabriele Puppis;
Article No. 169; pp. 169:1–169:18



Leibniz International Proceedings in Informatics
Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany



An important source of such operations are groups as the operation $(x, y, z) \mapsto xy^{-1}z$ with respect to some group satisfies these identities; such operations have been called *coset-generating polymorphisms* (terminology used in [15]; for the motivation for this name, see Lemma 9), which are also called *heaps* (see Section 6). For structures $\mathfrak{B}$ with a Maltsev polymorphism, the polynomial-time tractability of $\text{CSP}(\mathfrak{B})$ has been shown prior to [12, 34] and with a simpler algorithm (Bulatov and Dalmau [14]). But again, this algorithm is *not* a uniform polynomial-time algorithm. Nevertheless, the algorithm of Bulatov and Dalmau is *semiuniform* in the sense that, if the algorithm is not only given $\mathfrak{B}$ but also a Maltsev polymorphism of $\mathfrak{B}$, then it can answer the question whether there exists a homomorphism from $\mathfrak{A}$ to $\mathfrak{B}$ in polynomial time.¹ Chen and Larose [15] pointed out that a uniform polynomial-time algorithm for constraints with a Maltsev polymorphism would thus follow from the existence of a polynomial-time algorithm that takes a structure $\mathfrak{B}$ as input and computes a Maltsev polymorphism of $\mathfrak{B}$; this problem is called the *creation-metaproblem* (or *creation-metaquestion*, see [15]) for Maltsev polymorphisms. Conversely, if there exists a uniform polynomial-time algorithm for structures $\mathfrak{B}$ with a Maltsev polymorphism, then the creation-metaproblem for Maltsev polymorphisms is in P (a consequence of Theorem 4.7 in [15]; we use P also for function- P).

Some of the recently proposed candidates for a uniform polynomial-time algorithm for tractable CSPs were refuted using structures with a coset-generating polymorphism: *Datalog reductions to systems of linear equations over $\mathbb{Z}$* (a method proposed in [17]) fail to solve the CSP for a structure with the coset-generating polymorphism for the symmetric group S_{18} [25], and *singleton BLP+AIP* fails to solve the CSP for a structure with the coset-generating polymorphism for the dihedral group D_8 [33]. It is thus natural to determine the complexity of the creation-metaproblem for coset-generating polymorphisms (i.e., heaps) instead of Maltsev polymorphisms and the corresponding decision problem, which is simply called the *metaproblem* (or *metaquestion*). For the latter, the task is to decide whether the polymorphisms of a given finite structure $\mathfrak{B}$ (where the relations are given explicitly by lists of tuples) satisfy a certain polymorphism condition such as having a Maltsev polymorphism, or having a coset-generating polymorphism. And indeed, Chen and Larose [15, Section 8] ask: “Can anything be said about the complexity of deciding the presence of a coset-generating polymorphism?”. In this article we prove (in Section 7) that deciding whether a given structure has a coset-generating polymorphism is an NP -complete problem, thus answering the question of Chen and Larose.

► **Theorem 1.** *The following problem is NP -complete:*

Input: A finite structure $\mathfrak{B}$.

Question: Does $\mathfrak{B}$ have a coset-generating polymorphism?

While this does not rule out the existence of a uniform polynomial-time algorithm for CSPs with coset-generating polymorphisms, it shows that such a possible algorithm cannot use the approach to first compute the coset-generating polymorphism and then use Bulatov and Dalmau’s semiuniform algorithm [14].

One might ask whether the NP -hardness result in Theorem 1 still holds for coset-generating polymorphisms with respect to restricted classes of groups. Indeed, our proof only uses groups of order $4p$ for $p \geq 5$ prime – which, in particular, are all metabelian and, indeed, have an abelian normal subgroup of index 4.

¹ Also the polynomial-time algorithms of Zhuk and of Bulatov are semiuniform; instead of a Maltsev polymorphism, they work if a so-called *weak near unanimity polymorphism* of $\mathfrak{B}$ is part of the input.

Coset-generating operations f that come from *abelian* groups are called *abelian heaps*. If $\mathfrak{B}$ has a polymorphism that is an abelian heap, then a uniform polynomial-time algorithm is known (based on *affine integer programming* (AIP) introduced in [7]). We show that this algorithm can be used to compute in polynomial time for a given finite structure $\mathfrak{B}$ a Maltsev polymorphism when the existence of an abelian heap polymorphism is promised.

More generally, we introduce the *promise-metaproblem*, which is defined analogously to the *promise CSP* (for literature on this very active area, see, e.g., [2, 9]): if Σ_1 and Σ_2 are polymorphism conditions (formal definitions can be found in Section 2) such that Σ_1 implies Σ_2 , then $\text{PMeta}(\Sigma_1, \Sigma_2)$ is the problem of deciding for a given finite structure $\mathfrak{B}$ whether $\text{Pol}(\mathfrak{B})$ satisfies Σ_1 , or whether it does not even satisfy Σ_2 . For instance, if Σ_1 is the condition to have an abelian heap polymorphism, and Σ_2 is the condition to have a Maltsev polymorphism, then the argument above shows that $\text{PMeta}(\Sigma_1, \Sigma_2)$ is in P despite the fact that neither of the metaproblems for Σ_1 and for Σ_2 is known to be in P. More generally, we strengthen a result of Chen and Larose [15, Corollary 4.9] about the tractability of certain metaproblems and prove that $\text{PMeta}(\Sigma_1, \Sigma_2)$ is in P if

- Σ_1 is idempotent and there exists a uniform polynomial-time algorithm for $\text{CSP}(\mathfrak{B})$ for structures $\mathfrak{B}$ with $\text{Pol}(\mathfrak{B}) \models \Sigma_1$, and
- Σ_2 is a *linear strong Maltsev condition* (Proposition 7).²

Moreover, under the same conditions and provided that there exists a semiuniform polynomial-time algorithm for CSPs with polymorphisms satisfying Σ_2 , the uniform CSP for Σ_1 is in P if and only if the creation variant of the promise metaproblem is in P.³ In particular, we conclude that there is a uniform polynomial-time algorithm for CSPs with a coset-generating polymorphism if and only if the promise creation-metaproblem for coset-generating polymorphisms and Maltsev polymorphisms is in P.

Also one of the main hardness results by Chen and Larose about the metaproblem can be generalized to the larger class of promise metaproblems: we observe that if Σ_1 and Σ_2 are *non-trivial* and *consistent height one* strong Maltsev conditions (see Section 3 for definitions) such that Σ_1 implies Σ_2 , then $\text{PMeta}(\Sigma_1, \Sigma_2)$ is NP-complete (Proposition 6).

For a formal discussion of the complexity of uniform algorithms and metaproblems, we need to be more specific about how relational structures are given to a computer and how to measure the size of these representations. This can be found in Section 2. In Section 4 we introduce the promise metaproblem and promise creation-metaproblem and observe that a hardness result of Chen and Larose has a natural extension to the promise setting (Proposition 6). In Section 5 we discuss the connection between the promise metaproblem and uniform algorithms for the CSP, and prove a generalization of a known tractability result for metaproblems to promise metaproblems (Proposition 7). In Section 6 we give some background on coset-generating polymorphisms, and in Section 7 we prove our main result about the computational complexity of the metaproblem for coset-generating polymorphisms. We conclude and discuss open problems for future research in Section 9.

² A *strong Maltsev condition* is simply a finite set of height one conditions, see Section 2 for definitions. This is standard terminology, but we will avoid using it to prevent confusing with the Maltsev identities mentioned in the introduction. Linear strong Maltsev conditions are also called *minor conditions* in the Promise CSP literature.

³ Promise metaproblem should not be confused with meta-problems for promise CSPs, as studied e.g. in [24].

2 Structures, Homomorphisms, Polymorphisms

Let $k \in \mathbb{N}$ and $\tau \in \mathbb{N}^k$. A τ -structure $\mathfrak{A}$ consists of a set A and for every $i \in \{1, \dots, k\}$ a relation $R_i^{\mathfrak{A}} \subseteq A^{\tau_i}$. We refer to τ as the *signature* of $\mathfrak{A}$ (of size k). All τ -structures considered here will be *finite*, i.e., A is finite. If a τ -structure is considered as the input or output of some algorithm, we assume that its relations are represented by lists of tuples, i.e., the representation size of $\mathfrak{A}$ is $|A| + k + \sum_{i \in \{1, \dots, k\}} (\tau_i \cdot |R_i^{\mathfrak{A}}|)$. A *structure* is a τ -structure for some signature τ ; if we later write that a computational problem takes a structure as an input, we mean that τ and k are part of the input.

A *homomorphism* between two structures $\mathfrak{A}$ and $\mathfrak{B}$ with the same signature τ of size k is a function $h: A \rightarrow B$ such that for every $i \in \{1, \dots, k\}$ we have $(h(a_1), \dots, h(a_{\tau_i})) \in R_i^{\mathfrak{B}}$ whenever $(a_1, \dots, a_{\tau_i}) \in R_i^{\mathfrak{A}}$. If $\mathfrak{A}$ is a τ -structure and $m \in \mathbb{N}$, then $\mathfrak{B} := \mathfrak{A}^m$ is the τ -structure with domain A^m and

$$R_i^{\mathfrak{B}} := \{((a_1^1, \dots, a_1^m), \dots, (a_{\tau_i}^1, \dots, a_{\tau_i}^m)) \mid (a_1^j, \dots, a_{\tau_i}^j) \in R_i^{\mathfrak{A}} \text{ for all } j \in \{1, \dots, m\}\}.$$

A *polymorphism* of a structure $\mathfrak{B}$ is a homomorphism from $\mathfrak{B}^m$ to $\mathfrak{B}$, for some $m \in \mathbb{N}$. In other words, a polymorphism is an operation $B^m \rightarrow B$ for some m that, applied componentwise, preserves the relations of $\mathfrak{B}$. Note that the set of all polymorphisms of $\mathfrak{B}$ is a *clone*, i.e., it contains the projections and is closed under composition. A finite structure $\mathfrak{B}$ is called a *core* if every endomorphism⁴ of $\mathfrak{B}$ is an automorphism of $\mathfrak{B}$.

The problem of deciding whether there exists a homomorphism from a τ -structure $\mathfrak{A}$ to a τ -structure $\mathfrak{B}$ is NP-complete. This problem is sometimes called the *uniform CSP* (with both $\mathfrak{A}$ and $\mathfrak{B}$ being the input). If $\mathfrak{B}$ is a fixed τ -structure, then $\text{CSP}(\mathfrak{B})$ is the problem of deciding whether a given τ -structure $\mathfrak{A}$ has a homomorphism to $\mathfrak{B}$; this is often referred to as the *non-uniform CSP*. The finite-domain CSP dichotomy states that $\text{CSP}(\mathfrak{B})$ is in P if $\mathfrak{B}$ has a 4-ary *Siggers polymorphism*, i.e., a polymorphism s satisfying for all $a, r, e \in B$

$$s(a, r, e, a) = s(r, a, r, e),$$

and is NP-complete otherwise. This specific identity was found by Siggers [30], building on results from [4]. The result proved there also implies that if a structure has a Maltsev polymorphism, then it also has a 4-ary Siggers polymorphism. Note that unlike some older articles, we do not require that Siggers operations are idempotent. We also mention that for finite structures $\mathfrak{B}$ the following are equivalent.

- $\mathfrak{B}$ has a Siggers polymorphism;
- $\mathfrak{B}$ has a *cyclic polymorphism* [3], i.e., a polymorphism $c: B^k \rightarrow B$ for some $k \geq 2$ satisfying for all $x_1, \dots, x_k \in B$

$$c(x_1, \dots, x_k) = c(x_2, \dots, x_k, x_1).$$

- $\mathfrak{B}$ has a *weak near-unanimity polymorphism* [27], i.e., a polymorphism $w: B^k \rightarrow B$ for some $k \geq 2$ satisfying for all $x, y \in B$

$$w(y, x, \dots, x) = w(x, y, x, \dots, x) = \dots = w(x, \dots, x, y).$$

For every fixed pair of relational τ -structures $\mathfrak{B}$ and $\mathfrak{C}$ such that $\mathfrak{B}$ has a homomorphism to $\mathfrak{C}$, the *Promise CSP* for $(\mathfrak{B}, \mathfrak{C})$, denoted by $\text{PCSP}(\mathfrak{B}, \mathfrak{C})$, is defined as follows.

Input: A finite τ -structure $\mathfrak{A}$.

Output: “Yes” if $\mathfrak{A}$ has a homomorphism to $\mathfrak{B}$, “No” if $\mathfrak{A}$ has no homomorphism to $\mathfrak{C}$.

⁴ An endomorphism of $\mathfrak{B}$ is a homomorphism from $\mathfrak{B}$ to $\mathfrak{B}$, i.e., a unary polymorphism.

If $\mathfrak{A}$ has a homomorphism to $\mathfrak{C}$ but no homomorphism to $\mathfrak{B}$, then the algorithm solving $\text{PCSP}(\mathfrak{B}, \mathfrak{C})$ might answer arbitrarily. Despite a powerful theory that is available for studying the complexity of promise CSPs [2, 8, 9], the complexity of $\text{PCSP}(\mathfrak{B}, \mathfrak{C})$ has not been classified for all pairs $(\mathfrak{B}, \mathfrak{C})$. For instance, if K_ℓ is the clique with ℓ vertices, then the complexity of $\text{PCSP}(K_n, K_m)$ has already been studied in [18], and is conjectured to be NP-hard for all $3 \leq n < m$, but this remains open. There are problems of the form $\text{PCSP}(\mathfrak{A}, \mathfrak{B})$ in P where $\text{CSP}(\mathfrak{A})$ and $\text{CSP}(\mathfrak{B})$ are NP-hard [6].

3 Polymorphism Conditions

Let σ be a set of function symbols. A σ -identity is an expression of the form $s \approx t$ where s and t are σ -terms over some common set of variables $x_1, \dots, x_n$. Such an identity is called

- *linear* if each of the two terms contains at most one function symbol from σ ;
 - *height one* if exactly one symbol from σ occurs in each of the terms s and t .
- If Σ is a (not necessarily finite) set of σ -identities and $\mathcal{C}$ is a set of operations over B , then we write $\mathcal{C} \models \Sigma$ if we can interpret the function symbols from σ by operations from $\mathcal{C}$ such that for every identity $s \approx t$ from Σ and every instantiation of the variables in s and t by elements $b_1, \dots, b_n$ from B we have that $s(b_1, \dots, b_n) = t(b_1, \dots, b_n)$. Otherwise, we write $\mathcal{C} \not\models \Sigma$. If $\Sigma = \{s \approx t\}$ consists of a single identity, we also write $\mathcal{C} \models s \approx t$ instead of $\mathcal{C} \models \{s \approx t\}$. If Σ_1 and Σ_2 are sets of identities, we write $\Sigma_1 \models \Sigma_2$ if for every clone $\mathcal{C}$ with a finite domain we have that $\mathcal{C} \models \Sigma_2$ whenever $\mathcal{C} \models \Sigma_1$. A set of σ -identities Σ is called
- *idempotent* if Σ contains for every $f \in \sigma$ the identity $f(x, \dots, x) \approx x$;
 - *trivial* if $\text{Proj} \models \Sigma$, where Proj is the set of all projections over the set $\{0, 1\}$ (and *non-trivial* otherwise);
 - *consistent* if for every non-empty finite set D there exist idempotent operations on D that satisfy Σ (we use the definition from [15]; also see Lemma 3).

We say a function $g: D^k \rightarrow D$ is an *extension* of $f: C^k \rightarrow C$ for $C \subseteq D$ if $g|_{C^k} = f$ where $g|_{C^k}$ denotes the restriction of g to C^k .

► **Lemma 2.** *Let Σ be a set of linear σ -identities such that $\Sigma \not\models x \approx y$ and let $\mathcal{C}$ be a clone on some domain C such that $\mathcal{C} \models \Sigma$. Moreover, let $D = C \cup \{a\}$ for some $a \notin C$ and let $\mathcal{D}$ be the set of all operations that extend some $f \in \mathcal{C}$ (i.e., $\mathcal{D} = \{f: D^k \rightarrow D \mid k \in \mathbb{N}, \hat{f}|_{C^k} \in \mathcal{C}\}$). Then $\mathcal{D}$ is a clone and $\mathcal{D} \models \Sigma$.*

Proof. First, let us observe that $\mathcal{D}$ is a clone: Clearly, all projections are in $\mathcal{D}$. Moreover, if $\hat{f}, \hat{g}_1, \dots, \hat{g}_k \in \mathcal{D}$, then we have $f, g_1, \dots, g_k \in \mathcal{C}$ where $f, g_1, \dots, g_k$ denote the respective restrictions to C^ℓ for suitable ℓ . Hence, because $\mathcal{C}$ is a clone, also their composition is in $\mathcal{C}$ (which is the same as the restriction of $\hat{f}(\hat{g}_1, \dots, \hat{g}_k)$); hence, also $\hat{f}(\hat{g}_1, \dots, \hat{g}_k) \in \mathcal{D}$ and $\mathcal{D}$ is a clone.

To show that $\mathcal{D} \models \Sigma$, we define an equivalence relation $\equiv$ on the set of linear σ -terms over a countable set of variables $\mathcal{X}$. Note that a linear σ -term is either a single variable or a term of the form $f(x_1, \dots, x_n)$ where $f \in \sigma$ is an n -ary function symbol and $x_1, \dots, x_n \in \mathcal{X}$ (possibly with repetitions). We can also denote such a linear term in the form $s(y_1, \dots, y_k) = f(y_{i_1}, \dots, y_{i_n})$ where $y_1, \dots, y_k$ are pairwise distinct variables and $i_1, \dots, i_n \in \{1, \dots, k\}$.

We define $\equiv$ to be the smallest equivalence relation which contains Σ (recall that formally Σ is a set of pairs of linear σ -terms) and which is closed under substituting variables for other variables. Thus, in particular, we have $\Sigma \models \{s \approx t \mid s \equiv t\}$. Observe that

(1) as $\Sigma \not\models x \approx y$, we cannot have both $f(x_1, \dots, x_k) \equiv x_i$ and $f(x_1, \dots, x_k) \equiv x_j$ if $x_i \neq x_j$.

We may identify the function symbols in σ with operations in $\mathcal{C}$ that witness that $\mathcal{C}$ satisfies Σ . Using this identification, we further observe that,

- (2) if $s(y_1, \dots, y_k)$ is a linear term where $y_1, \dots, y_k \in \mathcal{X}$ are distinct variables and if $s(y_1, \dots, y_k) \equiv y_i$ for some $i \in \{1, \dots, k\}$, then $s(c_1, \dots, c_k) = c_i$ for all $c_1, \dots, c_k \in C$.

Now, for each $f \in \sigma$ identified with $f: C^k \rightarrow C$, let us describe an extension $\hat{f}: D^k \rightarrow D$ such that the resulting set of functions satisfies Σ . In order to do so, fix a map $\pi: D \rightarrow C$ which is the identity on C . For $f \in \sigma$ and $a_1, \dots, a_k \in D$ we define

$$\hat{f}(a_1, \dots, a_k) := \begin{cases} a_i & \text{if there exists an injective } \xi: \{a_1, \dots, a_k\} \rightarrow \mathcal{X} \\ & \text{such that } f(\xi(a_1), \dots, \xi(a_k)) \equiv \xi(a_i) \\ f(\pi(a_1), \dots, \pi(a_k)) & \text{otherwise.} \end{cases}$$

By (1), it follows that $\hat{f}$ is well-defined (note that the definition depends on π , but π is fixed – however, it does not depend on ξ , because the relation $\equiv$ is invariant under substitution of variables). Moreover, if $a_1, \dots, a_k \in C$, we have $\hat{f}(a_1, \dots, a_k) = f(a_1, \dots, a_k)$ (in the case that there exists an injective $\xi: \{a_1, \dots, a_k\} \rightarrow \mathcal{X}$ with $f(\xi(a_1), \dots, \xi(a_k)) \equiv \xi(a_i)$, we rely on (2)).

Finally, the collection of all the $\hat{f}$ for $f \in \sigma$ satisfies Σ . For identities involving $f(x_1, \dots, x_k)$ with $f(x_1, \dots, x_k) \equiv x_i$, this is because we defined $\hat{f}(a_1, \dots, a_k) = a_i$; for other identities, this follows immediately, because the set of all f 's (with $f \in \sigma$) already satisfies Σ . ◀

The purpose of the following lemma is to explain the concept of “consistency” for sets of linear identities; the second statement illustrates why we cannot drop the idempotence assumption in the first statement.

► **Lemma 3.** *An idempotent set of linear identities Σ is consistent if and only if $\Sigma \not\models x \approx y$. A set of height one identities Σ is consistent if and only if $\Sigma \not\models f(x) \approx g(y)$.*

Proof. Clearly, if $\Sigma \models x \approx y$, then all finite models of Σ have a domain of size one. Hence, if D has cardinality $n \geq 2$, then there is no set of operations on D that satisfies Σ , so Σ is not consistent. On the other hand, if $\Sigma \not\models x \approx y$, by Lemma 2, starting with a one-element domain, we find clones with arbitrary domain size that satisfy Σ . If Σ is idempotent, then we even find idempotent operations that satisfy Σ , so Σ is consistent.

To see the second statement, observe that if $\Sigma \models f(x) \approx g(y)$ and $\mathcal{C}$ is a clone such that $\mathcal{C} \models \Sigma$, then $\mathcal{C}$ must contain a constant unary operation, which can only be idempotent if the domain D of $\mathcal{C}$ has at most one element. So Σ is not consistent. Conversely, suppose that $\Sigma \not\models f(x) \approx g(y)$. It is known that in this case $\mathcal{D} \models \Sigma \cup \Delta$, where $\mathcal{D}$ is the clone of all idempotent operations on a two-element set D (Proposition 3.2.5 in [31]), and Δ are equations that express that all functions mentioned in Σ are idempotent. Let S be a finite superset of D ; then applying Lemma 2 sufficiently many times we obtain a clone on S which also satisfies $\Sigma \cup \Delta$. Hence, Σ is consistent. ◀

Note that the existence of a Siggers polymorphism can be expressed by the height one identity

$$f(a, r, e, a) \approx f(r, a, r, e)$$

and the existence of a Maltsev polymorphism can be expressed by the idempotent two-element set of linear identities

$$\Sigma_{\text{Maltsev}} := \{m(x, x, y) \approx y, m(y, x, x) \approx y\}. \quad (1)$$

Both of these sets of identities are non-trivial and consistent.

4 The Promise Metaproblem

If Σ is a set of σ -identities, then the *metaproblem for Σ* , denoted by $\text{Meta}(\Sigma)$, is the following computational problem.

Input: A finite structure $\mathfrak{B}$.

Output: “Yes” if $\text{Pol}(\mathfrak{B}) \models \Sigma$, “No” otherwise.

The following was shown by Chen and Larose [15].

► **Theorem 4** (Theorem 6.2 in [15]). *Let Σ be a non-trivial consistent finite set of height one identities. Then $\text{Meta}(\Sigma)$ is NP-complete.*

In particular, deciding whether $\mathfrak{B}$ has a Siggers polymorphism is NP-complete [15]. Note, however, that this theorem does not make any statement about deciding the existence of an idempotent Siggers polymorphism or the existence of a Maltsev polymorphism (since the idempotence of an operation cannot be expressed by a set of height one identities).

If $\mathfrak{B}$ is additionally promised to be a core, then it is easy to see that there exists a Siggers polymorphism if and only if there exists an idempotent Siggers polymorphism. We do not know whether there exists a polynomial-time algorithm to test whether $\mathfrak{B}$ has an idempotent Siggers polymorphism, and the same question is open for Maltsev polymorphisms.

Similarly to the case of the CSP, we can relax the problem to a promise version as follows. Let Σ_1 and Σ_2 be sets of σ -identities such that $\Sigma_1 \models \Sigma_2$. Then the *promise metaproblem for (Σ_1, Σ_2)* , denoted by $\text{PMeta}(\Sigma_1, \Sigma_2)$, is the following computational problem.

Input: A finite structure $\mathfrak{B}$.

Output: “Yes” if $\text{Pol}(\mathfrak{B}) \models \Sigma_1$, “No” if $\text{Pol}(\mathfrak{B}) \not\models \Sigma_2$.

Moreover, if the set $\tau \subseteq \sigma$ of symbols that appear in Σ_2 is finite, then the *promise creation-metaproblem for (Σ_1, Σ_2)* , denoted by $\text{PCMeta}(\Sigma_1, \Sigma_2)$, is the following computational problem.

Input: A finite structure $\mathfrak{B}$.

Output: A map $f: \tau \rightarrow \text{Pol}(\mathfrak{B})$ witnessing that $\text{Pol}(\mathfrak{B}) \models \Sigma_2$, if $\text{Pol}(\mathfrak{B}) \models \Sigma_1$.
If $\text{Pol}(\mathfrak{B}) \not\models \Sigma_1$, the output may be anything.

Thus, the output consists of an interpretation of each function symbol in τ . The following observation explains the name PCMeta.

► **Observation 5.** *If Σ_2 is finite and $\text{PCMeta}(\Sigma_1, \Sigma_2)$ is in P, then $\text{PMeta}(\Sigma_1, \Sigma_2)$ is in P.*

This follows simply by applying the algorithm for $\text{PCMeta}(\Sigma_1, \Sigma_2)$ and then testing whether the returned family of operations satisfies Σ_2 (this is indeed in P as Σ_2 is finite and not part of the input here). An inspection of Chen and Larose’s proof of Theorem 4 shows that it proves the following stronger result.

► **Proposition 6.** *Let Σ_1 and Σ_2 be finite sets of height one identities such that $\Sigma_1 \models \Sigma_2$, Σ_1 is consistent, and Σ_2 is non-trivial. Then $\text{PMeta}(\Sigma_1, \Sigma_2)$ is NP-hard.*

Note that we only state NP-hardness in Proposition 6, because formally NP only contains decision problems, not promise problems ($\text{Meta}(\Sigma_1)$, in contrast, is of course in NP).

Proof sketch. The result is shown by a reduction from graph 3-colorability. For a given finite graph G , Chen and Larose [15, Lemma 6.6] construct a relational structure $\mathfrak{B}$ which has the property that G is 3-colorable if and only if $\mathfrak{B}$ is not a core. Moreover, if G is 3-colorable, then the construction is such that the polymorphisms of the core of $\mathfrak{B}$ satisfy all consistent

idempotent sets of linear identities (see [15, Lemma 6.6 (2)]). Thus, the polymorphism clone of the core of $\mathfrak{B}$ and therefore also $\text{Pol}(\mathfrak{B})$ itself satisfy all consistent sets of height one identities. If G is not 3-colorable, then $\text{Pol}(\mathfrak{B})$ has the property that it does not satisfy any non-trivial height one identity (a consequence of [15, Fact 4]). This shows the statement. ◀

5 Connection with the Uniform CSP

The question whether certain tractability conditions for CSPs *uniformize* has been asked and studied by Kolaitis and Vardi [22]. Let Σ be a set of identities. The *uniform CSP* for Σ is defined as follows.

Input: A pair of finite structures $(\mathfrak{A}, \mathfrak{B})$ of the same signature such that $\text{Pol}(\mathfrak{B}) \models \Sigma$.

Output: “Yes” if there is a homomorphism from $\mathfrak{A}$ to $\mathfrak{B}$, “No” otherwise.

If $\mathfrak{B}$ does not satisfy Σ , then an algorithm for this problem might answer arbitrarily. If Σ is the set of identities that states the existence of a Maltsev (or Siggers, etc.) polymorphism, then the computational problem introduced above is called the *uniform CSP for Maltsev constraints* (or for *Siggers constraints*, etc.). It is an open problem whether there is a polynomial-time algorithm that solves the uniform CSP for Siggers constraints; this is stated as Problem 1 in Zhuk [32] (there, the problem is stated using weak near unanimity polymorphisms, which, however, is equivalent as explained in Section 2).

The following polynomial-time tractability result generalizes the second part of Theorem 4.7 in [15], which is the special case where $\Sigma_1 = \Sigma_2$. Be aware that Σ_1 is neither required to be finite nor linear.

► **Proposition 7.** *Let Σ_1 be an idempotent set of identities such that the uniform CSP for Σ_1 is in P, and let Σ_2 be a finite set of linear identities such that $\Sigma_1 \models \Sigma_2$. Then $\text{PCMeta}(\Sigma_1, \Sigma_2)$ and $\text{PMeta}(\Sigma_1, \Sigma_2)$ are in P.*

Proof. Let $\mathfrak{B}$ be a finite structure and let $\mathfrak{C}$ be the expansion of $\mathfrak{B}$ whose signature contains a symbol $R_=_$ for the equality relation and a symbol R_b for every element $b \in B$ which denotes the unary relation $\{b\}$. Note that if $\mathfrak{B}$ has polymorphisms satisfying Σ_1 , then, as they are idempotent, these polymorphisms are also polymorphisms of $\mathfrak{C}$. Therefore, $\mathfrak{C} \models \Sigma_1$. Since $\Sigma_1 \models \Sigma_2$ and each polymorphism of $\mathfrak{C}$ must be idempotent, it follows that $\Sigma_1 \models \Sigma_2 \cup \Delta$ and $\mathfrak{C} \models \Sigma_2 \cup \Delta$, where Δ is a set of equations that express that all functions mentioned in Σ_2 are idempotent. Conversely, if $\mathfrak{B} \not\models \Sigma_2$, then certainly $\mathfrak{B} \not\models \Sigma_2 \cup \Delta$ and $\mathfrak{C} \not\models \Sigma_2 \cup \Delta$. Therefore, from now on we can safely replace Σ_2 by $\Sigma_2 \cup \Delta$ and, hence, assume that Σ_2 is idempotent.

We may suppose without loss of generality that Σ_2 is the union of a set of height one identities and the identities of the form $f(x, \dots, x) \approx x$.

The following construction is essentially the *indicator structure* from [21]. For each function symbol of arity k that appears in Σ_2 , we create a copy of $\mathfrak{B}^k$ (i.e., the structure with domain B^k and the relations as defined in Section 2). Let $\mathfrak{J}$ be the structure obtained from the disjoint union over all these copies as follows. For every $b \in B$, we add to $R_b^{\mathfrak{J}}$ all tuples of the form $(b, \dots, b)$ in the copies of $\mathfrak{B}$. We additionally add for every identity $f(x_{i_1}, \dots, x_{i_k}) \approx g(x_{j_1}, \dots, x_{j_\ell})$ in Σ_2 and every function $\alpha: \{x_{i_1}, \dots, x_{i_k}, x_{j_1}, \dots, x_{j_\ell}\} \rightarrow B$ the constraint $c = d$ where $c = (\alpha(x_{i_1}), \dots, \alpha(x_{i_k}))$ belongs to the copy of $\mathfrak{B}^k$ created for f , and $d = (\alpha(x_{j_1}), \dots, \alpha(x_{j_\ell}))$ belongs to the copy of $\mathfrak{B}^\ell$ created for g (adding the constraint $c = d$ means we include (c, d) to $R_=_^{\mathfrak{J}}$).

Observe that if there is a homomorphism h from $\mathfrak{J}$ to $\mathfrak{C}$, then $\text{Pol}(\mathfrak{B}) \models \Sigma_2$: each restriction of h to a copy of $\mathfrak{B}^k$ is a polymorphism of $\mathfrak{B}$ of arity k , and the additional equality constraints ensure that the respective operations satisfy the given identities Σ_2 . Let

A denote the uniform polynomial-time algorithm for Σ_1 . Run A to determine whether there is a homomorphism $\mathcal{I} \rightarrow \mathcal{C}$. If A rejects, then return “No”. To see that this is the correct answer, suppose for contradiction that $\text{Pol}(\mathfrak{B}) \models \Sigma_1$. Then the answer of the algorithm A must be correct. Hence, there is no homomorphism from $\mathcal{I}$ to $\mathcal{C}$ and, therefore, $\text{Pol}(\mathfrak{B}) \not\models \Sigma_2$, a contradiction to the assumption that Σ_1 implies Σ_2 .

If A accepts, we proceed like for the well-known self-reduction of SAT. If A accepts, pick an element x from $\mathcal{I}$. For every $b \in B$, do the following:

- Add the constraint $R_b(x)$, i.e., add x to $R_b^{\mathcal{I}}$.
- Run A on the resulting instance.
- If A accepts, then proceed in the same way with another element of $\mathcal{I}$.

If for some variable, A rejects for all elements $b \in B$, then we return “No”. Note that this answer must be correct: if $\text{Pol}(\mathfrak{B}) \models \Sigma_1$, then it is impossible that we run into this case by the correctness of A if the promise is fulfilled. If $\text{Pol}(\mathfrak{B}) \not\models \Sigma_1$, then the answer “No” is correct.

Otherwise, eventually all elements x of $\mathcal{I}$ are contained in $R_b^{\mathcal{I}}$ for some $b \in B$. Thus, given the promise $\text{Pol}(\mathfrak{B}) \models \Sigma_1$, the map $x \mapsto b$ is a solution to $\text{PCMeta}(\Sigma_1, \Sigma_2)$.

To solve $\text{PMeta}(\Sigma_1, \Sigma_2)$, it remains to verify in polynomial time whether the map that sends x to b is a homomorphism from $\mathcal{I}$ to $\mathfrak{B}$ (see Observation 5). If no, return “No”, otherwise return “Yes”. These answers are clearly correct. ◀

A *semiuniform algorithm for the CSP for Σ* (where Σ is some set of identities) is an algorithm that takes as input a pair of finite structures $(\mathfrak{A}, \mathfrak{B})$ of the same signature τ such that $\text{Pol}(\mathfrak{B}) \models \Sigma$ together with a map $f: \tau \rightarrow \text{Pol}(\mathfrak{B})$ witnessing that $\text{Pol}(\mathfrak{B}) \models \Sigma$ and outputs whether there is a homomorphism from $\mathfrak{A}$ to $\mathfrak{B}$.

The following can be seen as a form of converse of Proposition 7: there, the existence of a uniform algorithm for the CSP for Σ_1 implies that $\text{PCMeta}(\Sigma_1, \Sigma_2)$ is in P; here, the assumption that $\text{PCMeta}(\Sigma_1, \Sigma_2)$ is in P, together with a polynomial-time semiuniform algorithm for Σ_2 , implies that the uniform algorithm for Σ_1 is in P.

► **Proposition 8.** *Let Σ_1 and Σ_2 be sets of σ -identities such that $\Sigma_1 \models \Sigma_2$ and the set $\tau \subseteq \sigma$ of symbols that appear in Σ_2 is finite. If $\text{PCMeta}(\Sigma_1, \Sigma_2)$ is in P and there is a polynomial-time semiuniform algorithm for the CSP for Σ_2 , then the uniform CSP for Σ_1 is in P.*

Proof. Given a pair $(\mathfrak{A}, \mathfrak{B})$ of structures of the same signature such that $\text{Pol}(\mathfrak{B}) \models \Sigma_1$. By assumption, we can compute $f: \tau \rightarrow \text{Pol}(\mathfrak{B})$ witnessing that $\text{Pol}(\mathfrak{B}) \models \Sigma_2$. We can then use the semiuniform algorithm for the CSP for Σ_2 to decide in polynomial time whether there is a homomorphism from $\mathfrak{A}$ to $\mathfrak{B}$. ◀

6 Coset-Generating Polymorphisms

A *coset-generating operation* on a set A is an operation $m: A^3 \rightarrow A$ that can be written as $m(x, y, z) = x \circ y^{-1} \circ z$ for some operations $\circ$ and $\cdot^{-1}$ such that $(A, \circ, \cdot^{-1}, e)$ forms a group for some element $e \in A$. In the following, we use juxtaposition instead of $\circ$. Let G be a group. We write $H \leq G$ if H is a subgroup of G . The following is well-known and the motivation why these operations are called *coset-generating*; for the convenience of the reader we give a proof.

► **Lemma 9.** *Let $k \in \mathbb{N}$. Then a relation $R \subseteq G^k$ is preserved by the operation $m: G^3 \rightarrow G$ defined by $m(x, y, z) = xy^{-1}z$ if and only if $R = gH$ for some $g \in G^k$ and $H \leq G^k$ (i.e., R is a coset of a subgroup of G^k).*

169:10 Coset-Generating Polymorphisms

Proof. Let H be a subgroup of G^k . Let $g \in G^k$ and $h_1, h_2, h_3 \in H$. As usual, we may apply m to elements of G^k componentwise; then

$$m(gh_1, gh_2, gh_3) = gh_1(gh_2)^{-1}gh_3 = gh_1h_2^{-1}h_3 \in gH$$

so m indeed preserves all cosets of H .

Conversely, suppose that $R \subseteq G^k$ is preserved by m . Choose $y \in R$ arbitrarily. We claim that $y^{-1}R$ is a subgroup H of G^k . This will show that $R = yH$ is a coset of a subgroup of G^k . Arbitrarily choose $a, b \in y^{-1}R$. Then $x := ya \in R$ and $z := yb \in R$. Hence, $m(x, y, z) = yay^{-1}yb = yab \in R$, so $ab \in y^{-1}R$ and $y^{-1}R$ is closed under the group operation. Moreover, $m(y, ya, y) = y(ya)^{-1}y = ya^{-1} \in R$, so $a^{-1} \in y^{-1}R$ and $y^{-1}R$ is also closed under taking inverses. $\blacktriangleleft$

A structure $\mathfrak{B}$ with a polymorphism which is a coset-generating operation is called a *coset structure* (with respect to G). CSPs for coset structures will be called *coset CSPs*. As $xx^{-1}y = y = yx^{-1}x$, every coset-generating operation is a Maltsev operation. Therefore, every coset structure has a Maltsev polymorphism, but the converse is not true (see, e.g., the Maltsev polymorphisms that appear in [5]), or the “No”-instances resulting from the reduction in Theorem 13. However, we have the following well-known fact.

► **Lemma 10.** *A relational structure $\mathfrak{B}$ is a coset structure if and only if it has a Maltsev polymorphism m that additionally satisfies for all $x, y, u, v, w \in B$ the following associativity identity*

$$m(u, x, m(v, y, w)) = m(m(u, x, v), y, w). \quad (2)$$

Proof. If $\mathfrak{B}$ is a coset structure and $f: (x, y, z) \mapsto xy^{-1}z$ is the coset-generating polymorphism of $\mathfrak{B}$, then f is a Maltsev operation that clearly satisfies the associativity condition (2). Conversely, if m is a Maltsev operation that satisfies (2), then we can pick an arbitrary element $e \in B$, and define $g: (x, y) \mapsto m(x, e, y)$. It is straightforward to verify that $(B; g)$ is a group with identity element e (the inverse of an element x is given by $m(e, x, e)$), and that m is the coset-generating polymorphism with respect to this group. $\blacktriangleleft$

Ternary Maltsev operations m that satisfy (2) are also called *heaps* [10] (the concept goes back to Prüfer [28] and Baer [1] in the 1920s). Thus, coset-generating polymorphisms (terminology used in [15]) and heaps are the same concept; in the following, we use both terms interchangeably, but prefer “coset-generating” in the context of polymorphisms, and “heaps” in the abstract setting of clones. Note that sometimes also the identity $m(u, x, m(v, y, w)) = m(u, m(y, v, x), w)$ is required for heaps. However, this is implied by the other identities, which follows from our Lemma 10 and can also be found in [11, Lemma 2.3].

We write Σ_{heap} for the identities that state the existence of a heap operation (i.e., (1) together with (2)). Using Proposition 7 and Proposition 8, we obtain the following corollary.

► **Corollary 11.** *PCMeta($\Sigma_{\text{heap}}, \Sigma_{\text{Maltsev}}$) is in P if and only if there is a uniform polynomial-time algorithm for coset CSPs.*

In the light of Proposition 6 and [15], it might be interesting to know whether Σ_{heap} can be enforced by some consistent set of linear identities. As we see next, this is not the case.

► **Proposition 12.** *Let Σ be a set of linear identities with $\Sigma \not\models x \approx y$. Then there is a structure $\mathfrak{B}$ without a coset-generating polymorphism such that $\text{Pol}(\mathfrak{B}) \models \Sigma$. In particular, being a coset structure cannot be defined by linear identities.*

Proof. Let $\mathfrak{A}$ be a structure with domain A such that $\text{Pol}(\mathfrak{A}) \models \Sigma$ and $n := |A| \geq 2$ (such an $\mathfrak{A}$ exists because $\Sigma \not\models x \approx y$). Define a structure $\mathfrak{B}$ with domain $B = A \cup \{b\}$ and with relations $R_i^{\mathfrak{B}} = R_i^{\mathfrak{A}}$; furthermore, add the unary relation $A \subseteq B$. Now, $\text{Pol}(\mathfrak{B})$ clearly consists of all extensions of polymorphisms of $\mathfrak{A}$ (meaning that the restriction of each $f \in \text{Pol}(\mathfrak{B})$ of arity k to A^k is in $\text{Pol}(\mathfrak{A})$). Therefore, by Lemma 2, $\text{Pol}(\mathfrak{B}) \models \Sigma$.

It remains to see that $\mathfrak{B}$ does not contain a coset-generating operation. Suppose otherwise. Then the corresponding group would have order $n + 1$. The size of each relation $R \subseteq B^r$ of $\mathfrak{B}$ needs to divide $(n + 1)^r$: the relation R must be a coset of a subgroup of a group with domain B^r by Lemma 9, and hence $|R|$ must divide $|B|^r$ by Lagrange's theorem. However, the unary relation A contains n elements, which is coprime to $(n + 1)^r$ – thus, $\text{Pol}(\mathfrak{B})$ cannot contain a coset-generating operation. $\blacktriangleleft$

7 NP-Hardness of Detecting Heap Polymorphisms

The following is our main result and answers the question of Chen and Larose [15] mentioned in the introduction.

► **Theorem 13** (see Theorem 1). *The following problem is NP-complete:*

Input: A finite structure $\mathfrak{B}$
Question: Is $\mathfrak{B}$ a coset structure?

► **Corollary 14.** *The creation-metaproblem for coset-generating polymorphisms is not in P unless $P = NP$.*

Proof. A polynomial-time algorithm for the creation-metaproblem can be used to solve the metaproblem in P by running the algorithm and then checking whether the output is indeed a coset-generating polymorphism. The latter can be done in polynomial time using the identities from Lemma 10 (see Observation 5). The statement therefore follows from Theorem 13. $\blacktriangleleft$

► **Remark 15.** By Corollary 14, if there exists a uniform polynomial-time algorithm for coset CSPs, it cannot use the approach of first computing the coset-generating polymorphism and then apply the semiuniform algorithm from [13].

Note that there still might be a uniform P-time algorithm for coset CSPs. Indeed, [15, Corollary 4.9] does not apply to this situation, because the property to be a coset structure cannot be defined by linear identities, as we have seen in Proposition 12.

Our proof of Theorem 13 is based on a reduction from the following graph problem, which has been shown to be NP-complete in [16, Theorem 4.9] (where the problem is called $(2, 1)$ graph coloring; see also [26]).

► **Lemma 16.** *The following problem is NP-complete:*

Input: A finite undirected simple graph $G = (V, E)$
Question: Can E be written as the disjoint union of a matching and the edge set of a bipartite graph?

In other words, the question in the computational problem is whether one can remove a matching so that the remaining graph becomes bipartite. For convenience of the reader, we give a proof in the appendix.

To reduce the computational problem from Lemma 16 to the question whether a given structure has a coset-generating polymorphism, we need some basic notions from group theory.

169:12 Coset-Generating Polymorphisms

Group basics. For $n \in \mathbb{N} \setminus \{0\}$, we write C_n for the cyclic group of order n . If G and H are groups, then we write $G \times H$ for the direct product of G and H . Given groups G, H and a homomorphism ϕ from H to the automorphism group $\text{Aut}(G)$ of G (or equivalently, an action of H on G via automorphisms) the semi-direct product $G \rtimes_\phi H$ is the group with elements $\{(g, h) \mid g \in G, h \in H\}$ and multiplication defined by

$$(g_1, h_1) \cdot (g_2, h_2) = (g_1 \phi(h_1)(g_2), h_1 h_2).$$

In this case, G (identified with $G \times \{1\}$) is a normal subgroup of $G \rtimes_\phi H$. Then the map π that sends each element f of $G \rtimes_\phi H$ to the coset fG of G is a group homomorphism $G \rtimes_\phi H \rightarrow (G \rtimes_\phi H)/G \cong H$. The map $\iota: h \mapsto (1, h)$ is an embedding of H into $G \rtimes_\phi H$ such that the composition $H \xrightarrow{\iota} G \rtimes_\phi H \xrightarrow{\pi} H$ is the identity (note that ι is not unique with this property). For simplicity, we simply write $G \rtimes H$ instead of $G \rtimes_\phi H$ and specify ϕ only verbally. For instance, if $G = C_p$ and $H = C_{2k}$, then $G \rtimes_\phi H$ with the action by inversion means that ϕ is the homomorphism from C_{2k} to $\text{Aut}(C_p)$ which sends the generator a of C_{2k} to the map $x \mapsto x^{-1} \in \text{Aut}(C_p)$.

Another example that is relevant later is $G = C_p$ for $p \equiv 1 \pmod{4}$ and again $H = C_4$, where $G \rtimes_\phi H$ with a faithful action means that ϕ is an injective homomorphism from C_4 to $\text{Aut}(C_p)$. This means that ϕ maps the generator a of C_4 to the map $x \mapsto x^k \in \text{Aut}(C_p)$ for $k = \pm(p-1)/4$. Note that the two different choices of k give isomorphic groups.

In order to prove our main theorem, we focus on dihedral groups. The dihedral group D_{2n} of order $2n$ for $n \geq 1$, is given by the following group presentation:

$$D_{2n} = \langle d, s \mid d^n = s^2 = 1, ds = sd^{-1} \rangle.$$

It is well-known that D_{2n} is isomorphic to the symmetry group of a regular n -gon and, thus, can be understood geometrically (see, e.g., [20, Theorem I.6.13]). Each element of D_{2n} has a unique normal form $s^k d^\ell$ with $k \in \{0, 1\}$ and $\ell \in \{0, \dots, n-1\}$. If $k = 1$, then $s^k d^\ell$ corresponds to a reflection, otherwise to a rotation. From this normal form, we can see that D_{2n} is isomorphic to the semidirect product $C_n \rtimes C_2$ with the action by inversion.

► **Lemma 17.** Let $m \in \mathbb{N}$. For the dihedral group $D_{4m} = \langle d, s \mid d^{2m} = s^2 = 1, ds = sd^{-1} \rangle$ the cosets of subgroups of order two are precisely as follows:

$$\{\{d^k, sd^\ell\} \mid k, \ell \in \{0, \dots, 2m-1\}\} \cup \{\{a, ad^m\} \mid a \in D_{4m}\}.$$

Proof. The lemma follows from the fact that the subgroups of D_{4m} of order two are the subgroups generated by sd^k for $k \in \{0, \dots, 2m-1\}$ (reflections at different axes) and the subgroup generated by d^m (rotation by 180 degrees). ◀

In particular, the graph $\Gamma = (V, E)$ defined by $V := D_{4m}$ and

$$E := \{\{u, v\} \mid \{u, v\} \text{ is a coset of a subgroup of } D_{4m}\}$$

can be written as the union of the complete bipartite graph $K_{2m, 2m}$ and a perfect matching on both sides.

► **Remark 18.** For an odd integer r , the corresponding graph for the dihedral group of order $2r$ is the complete bipartite graph $K_{r, r}$ (as the dihedral group then does not contain a rotation by 180 degrees).

Proof of Theorem 13. We reduce the problem from Lemma 16 to our problem.

Let $\Gamma = (V, E)$ be a given graph with $n = |V|$. We may assume without loss of generality that Γ has a vertex of degree at least four (otherwise, we can add such a vertex to Γ). First, compute a prime $p \geq 5$ with $n \leq 2p$ (this is clearly possible by checking whether any of the integers between $n/2$ and n is prime and by the Bertrand–Chebyshev theorem the existence of such a prime is guaranteed).

The domain of the structure $\mathfrak{B}$ will be $B := \{1, \dots, 4p\}$; we consider V as a subset of B . For the signature τ of $\mathfrak{B}$, it will be convenient to use the finite set E as the index set for the entries of the tuple τ , rather than $\{1, \dots, |E|\}$; define $\tau \in \mathbb{N}^E$ as $\tau := (1, \dots, 1)$. For $\{u, v\} \in E$, we have $R_{\{u, v\}}^{\mathfrak{B}} := \{u, v\}$ (thus, in particular, $\mathfrak{B}$ has only unary relations, each of which contains exactly two elements; moreover, the elements of $B \setminus V$ are not contained in any relation). Clearly, given Γ , the structure $\mathfrak{B}$ can be computed in logarithmic space. We claim that $\mathfrak{B}$ has a coset-generating polymorphism if and only if the edges of Γ can be partitioned into a bipartite graph and a matching.

First, assume that $\mathfrak{B}$ has a coset-generating polymorphism. Then, by Lemma 9, its relations are cosets of subgroups of some group G of order $4p$. It is well known⁵ that, since $p \geq 5$ is prime, there are up to isomorphism, at most five groups of order $4p$, namely

1. $C_2 \times C_2 \times C_p$,
2. $C_4 \times C_p \cong C_{4p}$,
3. $C_2 \times D_{2p} \cong D_{4p}$,
4. $C_p \rtimes C_4$ with the action by inversion (dicyclic group), and
5. possibly $C_p \rtimes C_4$ with a faithful action (if $p \equiv 1 \pmod{4}$).

Clearly, each of these groups has order $4p$; the fact that every group of order $4p$ is isomorphic to one of the groups in this list can be shown by elementary group theory (essentially the Sylow theorems; see, e.g. [23, Chapter 1, Section 6]). In particular, one first shows that in every such group the (unique) p -Sylow subgroup is normal (see [23, Exercise 28]).

In the abelian cases (Item 1 and 2) and in the case $G \cong C_p \rtimes C_4$ with the action by inversion (Item 4), there are at most three subgroups of order two; therefore, each element can be contained in at most three relations. Hence, these cases are ruled out, because we required that Γ has a vertex of degree four.

In the case $C_p \rtimes C_4$ with the faithful action (Item 5), the subgroups of order two are all contained in the unique subgroup isomorphic to D_{2p} (this is because under the projection $C_p \rtimes C_4 \rightarrow C_4$ an element of order two must map to the element of order two in C_4 ; thus, all order-two elements are contained in the unique subgroup of $C_p \rtimes C_4$ that is isomorphic to $C_p \rtimes C_2$). Therefore, according to Remark 18, Γ would have to be a subgraph of the disjoint union of two copies of $K_{p,p}$ (one for each coset of D_{2p}) – in particular, Γ is bipartite.

Therefore, the only possible group is D_{4p} . In this case, by Lemma 17, we know how the cosets of subgroups of order two look like. In particular, E can be written as the union of a bipartite graph and a matching.

Conversely, assume that Γ can be written as the union of a bipartite graph and a matching. Write $V = C \cup D$ where C and D are the two sides of the bipartite graph – meaning that the edges within C as well as the edges within D form a matching. Then we can choose an injection $f: V \rightarrow D_{4p}$ such that

- $f(C) \subseteq \langle d \rangle$,
- $f(D) \subseteq s\langle d \rangle$, and
- for every edge $\{u, v\}$ with both $u, v \in C$ or both $u, v \in D$ we have $f(u) = f(v)d^p$.

⁵ See e.g. the groupprops wiki https://groupprops.subwiki.org/wiki/Classification_of_groups_of_order_four_times_a_prime_congruent_to_1_modulo_4 (accessed on April 27, 2026)

This is possible because $|C|, |D| \leq |\langle d \rangle|$ and the edges within C and D form a matching. Hence, by Lemma 17, all relations of $\mathfrak{B}$ are cosets of subgroups of D_{4p} ; thus, $\mathfrak{B}$ has a coset-generating polymorphism. $\blacktriangleleft$

Observe that the structure $\mathfrak{B}$ in this proof *always* has a Maltsev polymorphism: indeed, any map $m: B^3 \rightarrow B$ satisfying the Maltsev identities leaves the defined relations invariant as they are all unary and contain exactly two elements.

8 Abelian Heap Polymorphisms

In the context of the metaproblem for coset-generating polymorphisms, Chen and Larose [15] also state “One can also ask these questions for restricted classes of groups.” In particular, the complexity of the following computational problem remains open.

Input: A structure $\mathfrak{B}$

Question: Is $\mathfrak{B}$ a coset structure for some abelian group?

It is easy to see that a structure $\mathfrak{B}$ is a coset structure for some abelian group if and only if it is preserved by a heap operation m which is *abelian*⁶, i.e., satisfies

$$m(x, y, z) \approx m(z, y, x).$$

We write Σ_{abheap} for the identities that state the existence of an abelian heap operation.

► **Corollary 19.** $\text{PMeta}(\Sigma_{\text{abheap}}, \Sigma_{\text{Maltsev}})$ is in P.

Proof. If $\mathfrak{B}$ has an abelian heap polymorphism, then by Lemma 10 it is preserved by $(x, y, z) \mapsto x - y + z$ with respect to some abelian group G . Hence, for every $n \geq 2$ it is also preserved by $(x_1, y_1, x_2, y_2, \dots, x_n) \mapsto x_1 - y_1 + x_2 - y_2 + \dots + x_n$, which is an example of a so-called *alternating polymorphism*. It is known that for such $\mathfrak{B}$, the AIP algorithm mentioned in the introduction is a uniform polynomial-time algorithm [2, Theorem 7.19]. Hence, the statement follows from Proposition 7. $\blacktriangleleft$

Note that $\text{PMeta}(\Sigma_{\text{abheap}}, \Sigma_{\text{Maltsev}})$ is in P despite the fact that the polynomial-time tractability of $\text{Meta}(\Sigma_{\text{abheap}})$ and of $\text{Meta}(\Sigma_{\text{Maltsev}})$ is open.

9 Conclusion and Open Problems

In this work we have introduced the promise version of the metaproblem in constraint satisfaction, and showed that several known results about the complexity of the metaproblem and connection to the existence of uniform polynomial-time CSP algorithms extend naturally to this larger setting. We identified a pair of polymorphism conditions (Σ_1, Σ_2) whose promise metaproblem is in P, but where the metaproblem for Σ_1 and the metaproblem for Σ_2 are not known to be in P. We then proved that the problem of deciding whether a given finite structure has a coset-generating (i.e., heap) polymorphism is NP-complete, solving an open problem of Chen and Larose [15].

Before the present work, there were several plausible scenarios how the question for the uniform complexity for coset CSPs might be settled:

⁶ It is well known and easy to show that abelian heap operations are abelian in the sense of universal algebra (we refrain from giving the definition, since we only need it in this footnote), and every heap operation which is abelian in the sense of universal algebra is an abelian heap operation; this justifies the terminology.

- (A) there could be a polynomial-time algorithm that given a finite structure promised to have a coset-generating polymorphism actually computes a corresponding group. If this is the case, then one obtains a uniform polynomial-time algorithm for such CSPs by using Bulatov and Dalmau’s semiuniform algorithm for CSPs with Maltsev constraints [14];
- (B) the group structure could be difficult to compute, yet one could compute in polynomial time a Maltsev polymorphism (and, again, the algorithm from [14] can be used to decide the CSP);
- (C) there could be no polynomial-time algorithm to compute a Maltsev polymorphism, but still coset CSPs could be uniformly decidable in polynomial time by another algorithm;
- (D) the uniform CSP for coset structures could be hard – here, besides being NP-complete, there are other hardness results imaginable: for instance, it might be hard for graph isomorphism or hard for group isomorphism.

In this work we ruled out possibility (A) by Corollary 14 (unless $P = NP$) and possibility (C) by Corollary 11. In particular, if there is a uniform polynomial-time algorithm for coset CSPs, then a Maltsev polymorphism can be computed in P , but no coset-generating polymorphism can be computed in general (unless $P = NP$).

We ask the following questions that are related to this work.

- Is there a uniform polynomial-time algorithm for coset CSPs?
- What is the answer to this question restricted to domains of size $4p$, where p is a prime? (This question is motivated by our proof of Theorem 1.)
- What is the complexity of deciding whether a given structure has an abelian heap polymorphism?
- What is the complexity of deciding whether a given structure has an abelian Maltsev polymorphism?

The complexity of the metaproblem and the uniform CSP for many natural finite sets of linear identities remains open. We wonder whether the class of (promise) metaproblems for fixed sets of identities is perhaps a candidate for a class that exhibits a complexity non-dichotomy.

References

- 1 Reinhold Baer. Zur Einführung des Scharbegriffs. *Journal für die reine und angewandte Mathematik*, 160:199–207, 1929.
- 2 Libor Barto, Jakub Bulín, Andrei A. Krokhin, and Jakub Opršal. Algebraic approach to promise constraint satisfaction. *J. ACM*, 68(4):28:1–28:66, 2021. doi:10.1145/3457606.
- 3 Libor Barto and Marcin Kozik. Absorbing subalgebras, cyclic terms and the constraint satisfaction problem. *Logical Methods in Computer Science*, 8/1(07):1–26, 2012. doi:10.2168/LMCS-8(1:7)2012.
- 4 Libor Barto, Marcin Kozik, and Todd Niven. The CSP dichotomy holds for digraphs with no sources and no sinks (a positive answer to a conjecture of Bang-Jensen and Hell). *SIAM Journal on Computing*, 38(5), 2009. doi:10.1137/070708093.
- 5 Manuel Bodirsky and Andrew Moorhead. Conservative Maltsev constraint satisfaction problems, 2025. doi:10.48550/arXiv.2505.11395.
- 6 Joshua Brakensiek and Venkatesan Guruswami. An algorithmic blend of LPs and ring equations for promise CSPs. In Timothy M. Chan, editor, *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019, San Diego, California, USA, January 6-9, 2019*, pages 436–455. SIAM, 2019. doi:10.1137/1.9781611975482.28.
- 7 Joshua Brakensiek and Venkatesan Guruswami. Symmetric polymorphisms and efficient decidability of promise CSPs. In Shuchi Chawla, editor, *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms, SODA 2020, Salt Lake City, UT, USA, January 5-8, 2020*, pages 297–304. SIAM, 2020. doi:10.1137/1.9781611975994.18.

- 8 Joshua Brakensiek and Venkatesan Guruswami. Promise constraint satisfaction: Algebraic structure and a symmetric boolean dichotomy. *SIAM Journal on Computing*, 50(6):1663–1700, 2021. doi:10.1137/19M128212X.
- 9 Joshua Brakensiek, Venkatesan Guruswami, Marcin Wrochna, and Stanislav Živný. The power of the combined basic linear programming and affine relaxation for promise constraint satisfaction problems. *SIAM J. Comput.*, 49(6):1232–1248, 2020. doi:10.1137/20M1312745.
- 10 Simion Breaz, Tomasz Brzeziński, Bernard Rybołowicz, and Paolo Saracco. Heaps of modules and affine spaces. *Annali di Matematica Pura ed Applicata (1923 -)*, 203(1):403–445, 2024.
- 11 Tomasz Brzeziński. Trusses: Paragons, ideals and modules. *Journal of Pure and Applied Algebra*, 224(6):106258, 2020.
- 12 Andrei A. Bulatov. A dichotomy theorem for nonuniform CSPs. In *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2017, Berkeley, CA, USA, October 15–17*, pages 319–330, 2017. doi:10.1109/FOCS.2017.37.
- 13 Andrei A. Bulatov and Víctor Dalmau. Towards a dichotomy theorem for the counting constraint satisfaction problem. In *Proceedings of the Annual Symposium on Foundations of Computer Science (FOCS)*, pages 562–574, 2003.
- 14 Andrei A. Bulatov and Víctor Dalmau. A simple algorithm for Mal’tsev constraints. *SIAM Journal on Computing*, 36(1):16–27, 2006. doi:10.1137/050628957.
- 15 Hubie Chen and Benoît Larose. Asking the metaquestions in constraint tractability. *TOCT*, 9(3):11:1–11:27, 2017. doi:10.1145/3134757.
- 16 Lenore Cowen, Wayne Goddard, and C. Esther Jesurum. Defective coloring revisited. *J. Graph Theory*, 24(3):205–219, 1997. doi:10.1002/(SICI)1097-0118(199703)24:3<205::AID-JGT2>3.0.CO;2-T.
- 17 Victor Dalmau and Jakub Opršal. Local consistency as a reduction between constraint satisfaction problems, 2023. arXiv:2301.05084.
- 18 M. R. Garey and D. S. Johnson. The complexity of near-optimal graph coloring. *J. ACM*, 23(1):43–49, January 1976. doi:10.1145/321921.321926.
- 19 Michael Garey and David Johnson. *A guide to NP-completeness*. CSLI Press, Stanford, 1978.
- 20 Thomas W. Hungerford. *Algebra*. Graduate texts in mathematics 73. Springer, New York, 1. print. edition, 1980.
- 21 Peter Jeavons, David Cohen, and Marc Gyssens. Closure properties of constraints. *Journal of the ACM*, 44(4):527–548, 1997. doi:10.1145/263867.263489.
- 22 Phokion G. Kolaitis and Moshe Y. Vardi. Conjunctive-query containment and constraint satisfaction. *J. Comput. Syst. Sci.*, 61(2):302–332, 2000. doi:10.1006/jcss.2000.1713.
- 23 Serge Lang. *Algebra*. Springer, 2002. Revised third edition.
- 24 Alberto Larrauri. Ineffectiveness for search and undecidability of PCSP meta-problems. In *66th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2025, Sydney, Australia, December 14–17, 2025*, pages 1313–1350. IEEE, 2025. doi:10.1109/FOCS63196.2025.00070.
- 25 Moritz Lichter and Benedikt Pago. Limitations of affine integer relaxations for solving constraint satisfaction problems. In *52nd International Colloquium on Automata, Languages, and Programming, ICALP 2025, Aarhus, Denmark, July 8–11, 2025*, volume 334 of *LIPIcs*, pages 166:1–166:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. doi:10.4230/LIPIcs.ICALP.2025.166.
- 26 Carlos V. G. C. Lima, Dieter Rautenbach, Uéverton S. Souza, and Jayme Luiz Szwarcfiter. Bipartizing with a matching. In *COCOA 2018, Proceedings*, volume 11346 of *Lecture Notes in Computer Science*, pages 198–213. Springer, 2018. doi:10.1007/978-3-030-04651-4_14.
- 27 Miklós Maróti and Ralph McKenzie. Existence theorems for weakly symmetric operations. *Algebra Universalis*, 59(3):463–489, 2008.
- 28 H. Prüfer. Theorie der Abelschen Gruppen. I. Grundeigenschaften. *Mathematische Zeitschrift*, 20:165–187, 1924. URL: <http://eudml.org/doc/167792>.
- 29 Thomas J. Schaefer. The complexity of satisfiability problems. In *Proceedings of the Symposium on Theory of Computing (STOC)*, pages 216–226, 1978. doi:10.1145/800133.804350.

- 30 Mark H. Siggers. A strong Mal'cev condition for varieties omitting the unary type. *Algebra Universalis*, 64(1):15–20, 2010. doi:10.1007/s00012-010-0082-3.
- 31 Albert Vucaj. *Clones over finite sets and Minor Conditions*. PhD thesis, Dresden University of Technology, Germany, 2023.
- 32 Dmitriy Zhuk. A proof of the CSP dichotomy conjecture. *J. ACM*, 67(5):30:1–30:78, 2020. doi:10.1145/3402029.
- 33 Dmitriy Zhuk. Singleton algorithms for the constraint satisfaction problem, 2025. doi:10.48550/arXiv.2509.18434.
- 34 Dmitriy N. Zhuk. A proof of CSP dichotomy conjecture. In *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2017, Berkeley, CA, USA, October 15-17*, pages 331–342, 2017. arXiv:1704.01914.

A

 Proof of Lemma 16

For convenience of the reader, we restate and give a proof of Lemma 16.

► **Lemma 20** (Lemma 16 restated). *The following problem is NP-complete:*

- Input:** A finite undirected simple graph $G = (V, E)$
Question: Can E be written as the disjoint union of a matching and the edge set of a bipartite graph?

Proof. Let us show a reduction from positive not-all-equal 3SAT [19, 29]. Consider an instance

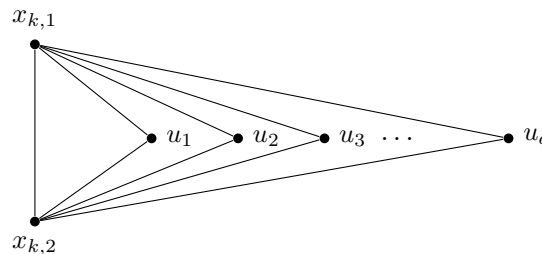
$$\Phi = \{\{L_{i,1}, L_{i,2}, L_{i,3}\} \mid i \in \{1, \dots, m\}\}$$

of this problem with m clauses and n variables $X_1, \dots, X_n$ (i.e., $L_{i,j} \in \{X_1, \dots, X_n\}$ for all $i \in \{1, \dots, m\}$ and $j \in \{1, 2, 3\}$). One may assume without loss of generality that each variable appears in at least three clauses (by simply duplicating clauses). We construct a graph (V, E) with vertex set

$$V = \{v_{i,j} \mid i \in \{1, \dots, m\}, j \in \{1, 2, 3\}\} \cup \{x_{k,\ell} \mid k \in \{1, \dots, n\}, \ell \in \{1, 2\}\}$$

(i.e. we have one vertex for each literal and two additional vertices for every variable) and edge set E such that

- $\{v_{i,1}, v_{i,2}, v_{i,3}\}$ becomes a clique for every $i \in \{1, \dots, m\}$,
- there is an edge $\{x_{k,1}, x_{k,2}\}$ for every $k \in \{1, \dots, n\}$,
- whenever $L_{i,j} = X_k$, there are the edges $\{v_{i,j}, x_{k,1}\}$ and $\{v_{i,j}, x_{k,2}\}$.



■ **Figure 1** For each variable X_k ($k \in \{1, \dots, n\}$) we have one of these subgraphs in the proof of Lemma 16. Here the vertices $u_1, \dots, u_q$ correspond to those vertices $v_{i,j}$ where the literal $L_{i,j}$ is the variable X_k . Note that $q \geq 3$.

Thus,

$$\begin{aligned} E = & \{ \{v_{i,j}, v_{i,k}\} \mid i \in \{1, \dots, m\}, j \neq k \in \{1, 2, 3\} \} \\ & \cup \{ \{x_{k,1}, x_{k,2}\} \mid k \in \{1, \dots, n\} \} \\ & \cup \{ \{v_{i,j}, x_{k,\ell}\} \mid i \in \{1, \dots, m\}, j \in \{1, 2, 3\}, k \in \{1, \dots, n\}, \ell \in \{1, 2\} \text{ with } L_{i,j} = X_k \}. \end{aligned}$$

The edges of the second and third type associated to a single variable X_k are illustrated in Figure 1 – the edges of the first type connect these respective gadgets associated to different variables. Clearly, this graph can be computed in polynomial time from the given instance Φ .

We claim that Φ has a solution if and only if the given graph can be written as a disjoint union of a matching and the edge set of a bipartite graph. In order to see this, first note that each of the cliques $\{v_{i,1}, v_{i,2}, v_{i,3}\}$ can be written as a union of a bipartite graph and a matching (e.g., the edges $\{v_{i,j}, v_{i,2}\}$ for $j \in \{1, 3\}$ form a bipartite graph, and the edges $\{v_{i,1}, v_{i,3}\}$ form a matching). Moreover, any such edge decomposition must have one vertex of each triangle on one side of the bipartite graph and the other two vertices on the other side (and there is precisely one matching edge).

Now assume that Φ has a satisfying assignment $\sigma: \{X_1, \dots, X_n\} \rightarrow \{0, 1\}$ (i.e., for every $i \in \{1, \dots, m\}$ we have $1 \leq \sigma(L_{i,1}) + \sigma(L_{i,2}) + \sigma(L_{i,3}) \leq 2$). Setting

$$A = \{v_{i,j} \mid \sigma(L_{i,j}) = 1\} \cup \{x_{k,\ell} \mid \sigma(X_k) = 0\}$$

and $B = V \setminus A$ gives us a bipartite decomposition of the graph after removing the matching consisting of all the edges $\{x_{k,1}, x_{k,2}\}$ for $k \in \{1, \dots, n\}$ and $\{v_{i,j}v_{i,k}\}$ whenever $\sigma(L_{i,j}) = \sigma(L_{i,k})$ with $i \in \{1, \dots, m\}$ and $j \neq k \in \{1, 2, 3\}$.

To see the other direction of the correctness proof, observe that in any decomposition of E into a matching and bipartite graph, all the edges $\{x_{k,1}, x_{k,2}\}$ must be part of the matching. This is because by our assumption $x_{k,1}$ and $x_{k,2}$ have at least three common neighbours, see Figure 1. (Each of the respective triangles containing the edge $\{x_{k,1}, x_{k,2}\}$ must contain one matching edge. Since there are at least three of these triangles, the only possibility is the edge $\{x_{k,1}, x_{k,2}\}$.)

Now, let $V = A \cup B$ be a bipartite decomposition of V after removing a suitable matching. From the previous observation, we conclude that if $L_{i,j} = L_{\mu,\nu}$, then $v_{i,j}$ and $v_{\mu,\nu}$ are either both in A or both in B , because they are connected by a path of length two that cannot contain a matching edge. Therefore, the assignment $\sigma: \{X_1, \dots, X_n\} \rightarrow \{0, 1\}$ with $\sigma(X_k) = 1$ if and only if $x_{k,1} \in A$ is a satisfying assignment. ◀